2025

Programming Fundamentals Lab 8

Lab Tasks: Task 4 to Task 10

NAME: MUHAMMAD FARHAN
ROLL NO: 25K-0538
SECTION: BCS-1K
PF LAB 8, LAB TASKS 4 TO 7.
COURSE INSTRUCTOR: SIR SHEERAZ IQBAL

Task 4: Upper and Lower Triangular Matrix Check.

```
#include <stdio.h>
int main()
{
   int matrix[3][3], upper=0, lower=0;
    for (int i = 0; i < 3; i++)
   {
      for (int j = 0; j < 3; j++)
      {
         printf("Enter the element of (%d)row, (%d)column of the matrix.\n", i+1, j+1);
         scanf("%d", &matrix[i][j]);
         if (i>j && matrix[i][j]==0)
            upper++;
         if (i<j && matrix[i][j]==0)
            lower++;

      }

   }
   if (upper==3 && lower==3)
      printf("\n\nThe input matrix is a Both a Upper and Lower Triangular Matrix.\n");
   else if (upper==3)
      printf("\n\nThe input matrix is a Upper Triangular Matrix.\n");
   else if (lower==3)
      printf("\n\nThe input matrix is a Lower Triangular Matrix.\n");
```

```
    else
        printf("\n\nThe input matrix is not a Triangular Matrix.\n");
return 0;
}
```

```
Enter the element of (1)row, (1)column of the matrix.
1
Enter the element of (1)row, (2)column of the matrix.
2
Enter the element of (1)row, (3)column of the matrix.
3
Enter the element of (2)row, (1)column of the matrix.
0
Enter the element of (2)row, (2)column of the matrix.
5
Enter the element of (2)row, (3)column of the matrix.
6
Enter the element of (3)row, (1)column of the matrix.
0
Enter the element of (3)row, (2)column of the matrix.
0
Enter the element of (3)row, (3)column of the matrix.
9


The input matrix is a Upper Triangular Matrix.
```

```
Enter the element of (1)row, (1)column of the matrix.
1
Enter the element of (1)row, (2)column of the matrix.
0
Enter the element of (1)row, (3)column of the matrix.
0
Enter the element of (2)row, (1)column of the matrix.
4
Enter the element of (2)row, (2)column of the matrix.
5
Enter the element of (2)row, (3)column of the matrix.
0
Enter the element of (3)row, (1)column of the matrix.
1
Enter the element of (3)row, (2)column of the matrix.
2
Enter the element of (3)row, (3)column of the matrix.
3


The input matrix is a Lower Triangular Matrix.
```

```
Enter the element of (1)row, (1)column of the matrix.
1
Enter the element of (1)row, (2)column of the matrix.
0
Enter the element of (1)row, (3)column of the matrix.
0
Enter the element of (2)row, (1)column of the matrix.
0
Enter the element of (2)row, (2)column of the matrix.
5
Enter the element of (2)row, (3)column of the matrix.
0
Enter the element of (3)row, (1)column of the matrix.
0
Enter the element of (3)row, (2)column of the matrix.
0
Enter the element of (3)row, (3)column of the matrix.
9


The input matrix is a Both a Upper and Lower Triangular Matrix.
```

Task 5: Diamond Pattern of Stars

Note: For this Tasks, I have made 2 different codes for the diamond pattern.
The end results of both are diamond pattern made of stars, but the arrangement of stars inside them are different.

Version 1:

```
#include <stdio.h>

int main()

{

   int k=0, rows, totalrows;

   printf("Enter the number of rows of the top half of the diamond.\n");

   scanf("%d", &rows);

   totalrows=rows*2-1;

   for (int i=1; i<=totalrows; i++)

   {

      i<=rows?k++:k--;

      for(int j=1; j<=totalrows; j++)

      {

         if (j>=rows+1-k && j<=rows-1+k)

         {

            printf("*");

         }

         else

         {

            printf(" ");

         }


      }
```

```
        printf("\n");
    }
return 0;
}
```

```
Enter the number of rows of the top half of the diamond.
5
    *
   ***
  *****
 *******
*********
 *******
  *****
   ***
    *
```

Version 2:

```
#include <stdio.h>
int main()
{
    int rows, totalrows, spaces, stars;
    printf("Enter the number of rows for the upper half of the diamond.\n");
    scanf("%d", &rows);
    printf("\n");
    totalrows=rows*2-1;
    spaces=rows-1;
    for (int i = 1; i <= totalrows; i++)
    {
        i<=rows?stars=i:stars--;
        for (int s=1;s<=spaces;s++)
        {
```

```
        printf(" ");
    }
    for (int j = 1; j <= stars ; j++)
    {
        printf("* ");
    }
    printf("\n");
    i<rows?spaces--:spaces++;
  }
return 0;
}
```

```
Enter the number of rows for the upper half of the diamond.
5

    *
   * *
  * * *
 * * * *
* * * * *
 * * * *
  * * *
   * *
    *
```

Task 6: Transpose of a Matrix

```
#include <stdio.h>
int main()
{
   int matrix[3][3], transpose[3][3];
   for (int i = 0; i < 3; i++)
   {
      for (int j = 0; j < 3; j++)
      {
         printf("Enter the element of (%d)row, (%d)column of the matrix.\n", i+1, j+1);
         scanf("%d", &matrix[i][j]);
         transpose[j][i]=matrix[i][j];
      }

   }
   printf("\nThe Original Matrix\n");
   for (int i = 0; i < 3; i++)
    {
      for (int j = 0; j < 3; j++)
      {
        printf("%d\t", matrix[i][j]);
      }
   printf("\n\n");
   }
   printf("\nTranspose of the Matrix\n");
```

```
    for (int i = 0; i < 3; i++)
     {
        for (int j = 0; j < 3; j++)
        {
           printf("%d\t", transpose[i][j]);
        }
    printf("\n\n");
    }
return 0;
}
```

```
Enter the element of (1)row, (1)column of the matrix.
1
Enter the element of (1)row, (2)column of the matrix.
2
Enter the element of (1)row, (3)column of the matrix.
3
Enter the element of (2)row, (1)column of the matrix.
4
Enter the element of (2)row, (2)column of the matrix.
5
Enter the element of (2)row, (3)column of the matrix.
6
Enter the element of (3)row, (1)column of the matrix.
7
Enter the element of (3)row, (2)column of the matrix.
8
Enter the element of (3)row, (3)column of the matrix.
9

The Original Matrix
1       2       3

4       5       6

7       8       9


Transpose of the Matrix
1       4       7

2       5       8

3       6       9
```

Task 7: Multiply Two Matrices

```
#include <stdio.h>
int main()
{
    int size;
    printf("Enter 2 for a 2x2 Matrix or 3 for a 3x3 Matrix.\n");
    scanf("%d", &size);
    int matrix1[size][size], matrix2[size][size], matrix3[size][size];
    for (int i = 0; i < size; i++)
    {
        for (int j = 0; j < size; j++)
        {
            matrix3[i][j]=0;
        }

    }
    printf("\n");
    for (int i = 0; i < size; i++)
    {
        for (int j = 0; j < size; j++)
        {
            printf("Enter the element of (%d)row, (%d)column of the first matrix.\n", i+1, j+1);
            scanf("%d", &matrix1[i][j]);
        }
```

```
}
printf("\n");
for (int i = 0; i < size; i++)
{
    for (int j = 0; j < size; j++)
    {
        printf("Enter the element of (%d)row, (%d)column of the second matrix.\n", i+1, j+1);
        scanf("%d", &matrix2[i][j]);
    }


}
printf("\n");
for (int i=0; i < size; i++)
{
    for (int j = 0; j < size; j++)
    {
        for (int k = 0; k < size; k++)
        {
            matrix3[i][j]+=matrix1[i][k]*matrix2[k][j];
        }


    }
}
printf("Multiplication of both Matrices\n");
for (int i = 0; i < size; i++)
{
```

```
        for (int j = 0; j < size; j++)
        {
            printf("%d\t", matrix3[i][j]);
        }
        printf("\n\n");
    }


return 0;
}
```

```
Enter 2 for a 2x2 Matrix or 3 for a 3x3 Matrix.
2

Enter the element of (1)row, (1)column of the first matrix.
1
Enter the element of (1)row, (2)column of the first matrix.
2
Enter the element of (2)row, (1)column of the first matrix.
3
Enter the element of (2)row, (2)column of the first matrix.
4

Enter the element of (1)row, (1)column of the second matrix.
1
Enter the element of (1)row, (2)column of the second matrix.
2
Enter the element of (2)row, (1)column of the second matrix.
3
Enter the element of (2)row, (2)column of the second matrix.
4

Multiplication of both Matrices
7       10

15      22
```

```
Enter 2 for a 2x2 Matrix or 3 for a 3x3 Matrix.
3

Enter the element of (1)row, (1)column of the first matrix.
1
Enter the element of (1)row, (2)column of the first matrix.
2
Enter the element of (1)row, (3)column of the first matrix.
3
Enter the element of (2)row, (1)column of the first matrix.
4
Enter the element of (2)row, (2)column of the first matrix.
5
Enter the element of (2)row, (3)column of the first matrix.
6
Enter the element of (3)row, (1)column of the first matrix.
7
Enter the element of (3)row, (2)column of the first matrix.
8
Enter the element of (3)row, (3)column of the first matrix.
9

Enter the element of (1)row, (1)column of the second matrix.
1
Enter the element of (1)row, (2)column of the second matrix.
2
Enter the element of (1)row, (3)column of the second matrix.
3
Enter the element of (2)row, (1)column of the second matrix.
4
Enter the element of (2)row, (2)column of the second matrix.
5
Enter the element of (2)row, (3)column of the second matrix.
6
Enter the element of (3)row, (1)column of the second matrix.
7
Enter the element of (3)row, (2)column of the second matrix.
8
Enter the element of (3)row, (3)column of the second matrix.
9

Multiplication of both Matrices
30      36      42

66      81      96
```

```
Multiplication of both Matrices
30      36      42

66      81      96

102     126     150
```

Task 8: Analyze Matrix Elements — Count Even, Odd, Positive, Negative, and Zeros

```
#include <stdio.h>
int main()
{
   int matrix[3][3];
   int counteven=0, countodd=0, countzero=0, countneg=0, countpos=0;
   for (int i = 0; i < 3; i++)
   {
      for (int j = 0; j < 3; j++)
      {
         printf("Enter the element of (%d)row, (%d)column of the matrix.\n", i+1, j+1);
         scanf("%d", &matrix[i][j]);
         if (matrix[i][j]>0)     countpos++;
         if (matrix[i][j]%2!=0)  countodd++;
         if (matrix[i][j]%2==0)  counteven++;
         if (matrix[i][j]==0)    countzero++;
         if (matrix[i][j]<0)     countneg++;
      }

   }
   printf("\n");
printf("There are %d positive elements in the matrix.\n", countpos);
printf("There are %d negative elements in the matrix.\n", countneg);
printf("There are %d zero elements in the matrix.\n", countzero);
printf("There are %d even elements in the matrix.\n", counteven);
```

```
printf("There are %d odd elements in the matrix.\n", countodd);

return 0;

}
```

```
Enter the element of (1)row, (1)column of the matrix.
5
Enter the element of (1)row, (2)column of the matrix.
-10
Enter the element of (1)row, (3)column of the matrix.
-11
Enter the element of (2)row, (1)column of the matrix.
0
Enter the element of (2)row, (2)column of the matrix.
90
Enter the element of (2)row, (3)column of the matrix.
63
Enter the element of (3)row, (1)column of the matrix.
20
Enter the element of (3)row, (2)column of the matrix.
0
Enter the element of (3)row, (3)column of the matrix.
-13

There are 4 positive elements in the matrix.
There are 3 negative elements in the matrix.
There are 2 zero elements in the matrix.
There are 5 even elements in the matrix.
There are 4 odd elements in the matrix.
```

Task 9: Rotate a Matrix by 90 Degrees Clockwise

```
#include <stdio.h>
int main()
{
   int matrix[3][3], transposerotate[3][3], tempo, counta=0;
   for (int i = 0; i < 3; i++)
   {
      for (int j = 0; j < 3; j++)
      {
         printf("Enter the element of (%d)row, (%d)column of the 3x3 matrix.\n", i+1, j+1);
         scanf("%d", &matrix[i][j]);
         transposerotate[j][i]=matrix[i][j];
      }

   }
   for (int i = 0; i < 3; i++)
   {
      tempo=transposerotate[i][0];
      transposerotate[i][0]=transposerotate[i][2];
      transposerotate[i][2]=tempo;
      for (int j = 0; j < 3; j++)
      {
         if (transposerotate[i][j]==matrix[i][j])
         {
            counta++;
```

```
            }
        }

    }
     printf("\nThe Original Matrix\n");
    for (int i = 0; i < 3; i++)
     {
        for (int j = 0; j < 3; j++)
        {
            printf("%d\t", matrix[i][j]);
        }
    printf("\n\n");
    }
    printf("\nRotated of the Matrix\n");
    for (int i = 0; i < 3; i++)
     {
        for (int j = 0; j < 3; j++)
        {
            printf("%d\t", transposerotate[i][j]);
        }
    printf("\n\n");
    }
    if (counta==9)  printf("The matrix's rotation is same as the original matrix.\n");
return 0;
}
```

```
Enter the element of (1)row, (1)column of the 3x3 matrix.
1
Enter the element of (1)row, (2)column of the 3x3 matrix.
2
Enter the element of (1)row, (3)column of the 3x3 matrix.
3
Enter the element of (2)row, (1)column of the 3x3 matrix.
4
Enter the element of (2)row, (2)column of the 3x3 matrix.
5
Enter the element of (2)row, (3)column of the 3x3 matrix.
6
Enter the element of (3)row, (1)column of the 3x3 matrix.
7
Enter the element of (3)row, (2)column of the 3x3 matrix.
8
Enter the element of (3)row, (3)column of the 3x3 matrix.
9

The Original Matrix
1       2       3

4       5       6

7       8       9


Rotated of the Matrix
7       4       1

8       5       2

9       6       3
```

```
Enter the element of (1)row, (1)column of the 3x3 matrix.
0
Enter the element of (1)row, (2)column of the 3x3 matrix.
1
Enter the element of (1)row, (3)column of the 3x3 matrix.
0
Enter the element of (2)row, (1)column of the 3x3 matrix.
1
Enter the element of (2)row, (2)column of the 3x3 matrix.
0
Enter the element of (2)row, (3)column of the 3x3 matrix.
1
Enter the element of (3)row, (1)column of the 3x3 matrix.
0
Enter the element of (3)row, (2)column of the 3x3 matrix.
1
Enter the element of (3)row, (3)column of the 3x3 matrix.
0

The Original Matrix
0       1       0

1       0       1

0       1       0


Rotated of the Matrix
0       1       0

1       0       1

0       1       0

The matrix's rotation is same as the original matrix.
```

Task 10: Pascal’s Triangle Pattern

```
#include <stdio.h>
int main()
{
    int nfactorial=1, nrfactorial=1, rfactorial=1, ncr, nr, rows;
    printf("Enter the number of rows for the Pascal Triangle.\n");
    scanf("%d", &rows);
    printf("\n");
    for (int i = 0; i < rows; i++) //printing number of rows
    {
        nfactorial=1; //resetting n! to 1 for fresh value of n! for each row
        for (int n = 1; n <= i; n++) //calculating n!
        {
            nfactorial*=n;
        }
        for (int k = rows-1-i; k >0; k--) //printing spaces for centering, which are totalrows-1
        {
            printf(" ");
        }
        for (int j = 0; j <= i ; j++)
        {
            nr=i-j; //calculating (n-r)
            for (int r=1; r <= nr; r++) //calculating (n-r)!
            {
                nrfactorial*=r;
```

```
        }
        for (int r=1; r <= j; r++) //calculating r!
        {
            rfactorial*=r;
        }

        ncr=nfactorial/(rfactorial*nrfactorial); //calulating nCr
        printf("%d ", ncr); //printing nCr
        nrfactorial=1; //reseting (n-r)! and r! to 1 for a fresh calculation for next element
        rfactorial=1;
    }
    printf("\n"); //going to next line
  }
return 0;
}
```

```
Enter the number of rows for the Pascal Triangle.
5

    1
   1 1
  1 2 1
 1 3 3 1
1 4 6 4 1
```

```
Enter the number of rows for the Pascal Triangle.
4

   1
  1 1
 1 2 1
1 3 3 1
```